



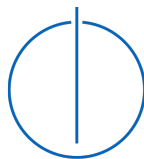
DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics: Games Engineering

Hardware-accelerated Raytracing in Dynamic Environments using Vulkan

Christian Mulser





DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics: Games Engineering

Hardware-accelerated Raytracing in Dynamic Environments using Vulkan

Hardwarebeschleunigtes Raytracing in dynamischen Umgebungen mittels Vulkan

Author:	Christian Mulser
Supervisor:	Prof. Dr. Rüdiger Westermann
Advisor:	Prof. Dr. Rüdiger Westermann
Submission Date:	September 28, 2025



I confirm that this bachelor's thesis in informatics: games engineering is my own work and I have documented all sources and material used.

Munich, September 28, 2025

Christian Mulser

Acknowledgments

I thank my supervisor, Rüdiger Westermann, for guiding me along the way and providing useful feedback to my work. I also thank Sebastian Wohner for providing access to the required modern RTX-GPUs and for quickly helping me out with technical difficulties.

Abstract

Contents

Acknowledgments	iii
Abstract	iv
1. Introduction	1
2. Related Work	3
2.1. Real-Time Ray Tracing on GPUs	3
2.2. Dynamic Scene Ray Tracing	3
2.3. Acceleration Structures and Memory Management	3
2.4. Summary and Research Gap	3
3. Technologies	4
3.1. System Information	4
3.2. Build Tools	4
3.3. Graphics API	4
3.4. Programming Language	4
3.5. File Formats	5
3.5.1. glTF	5
3.5.2. CSV	5
3.6. Libraries	5
3.6.1. Volk	5
3.6.2. VMA (Vulkan Memory Allocator)	5
3.6.3. vk-bootstrap	6
3.6.4. cgltf	6
3.6.5. CLI11	6
4. Implementation	7
4.1. Project Setup	7
4.1.1. Source Code Organization	7
4.2. Vulkan Abstraction	8
4.2.1. Move Semantics	8
4.2.2. Shared Pointers	9
4.2.3. The Builder Pattern	9
4.2.4. Basic Vulkan Object Structure	10
4.2.5. The Vulkan Context	11
4.2.6. The Command Manager	12

4.2.7. The VMA Allocator	13
4.2.8. Command Buffers	13
4.2.9. Resource Objects	15
4.2.10. Synchronization Objects	17
4.2.11. Pipeline-related Objects	18
4.2.12. Raytracing-related Objects	19
4.3. Scene	19
4.3.1. glTF Structure	19
4.3.2. Scene Structure	20
4.3.3. Updating a Scene	24
5. Results	28
6. Discussion	29
7. Conclusion	30
A. General Addenda	31
A.1. Detailed Addition	31
B. Figures	32
B.1. Example 1	32
B.2. Example 2	32
List of Figures	33
List of Tables	34
Bibliography	35

1. Introduction

For a long time, **rasterization** has been the standard for rendering 3D scenes in real-time (e.g. in video games). The basic steps of rasterization are as follows:

1. Every vertex of a 3D object gets transformed in the vertex shader.
2. The transformed triangles are projected onto a 2D pixel grid. For every pixel that the triangle covers, a fragment is generated.
3. The fragment shader runs for every generated fragment. It uses the data of the fragment as well as some global data (e.g. light sources) to determine the final color of the pixel.

Rasterization is very fast and efficient, but it does have its downsides as well. When determining the color of a pixel, we only have access to the data in the current fragment and some global data. The fragment is not aware of other objects in the scene. This makes effects like shadows and reflections very difficult and they require a workaround like shadow-mapping and screen-space-reflections. In recent years, a new way to do real-time rendering slowly emerged, which of course is **ray tracing**.

Ray tracing works by casting a ray through the pixels of your screen into the 3D scene, to determine the surface to be displayed. When you hit a surface, you can cast secondary rays such as shadow rays, reflection rays or refraction rays. This is much closer to real life, where light rays bounce between objects until they reach our eyes or a camera lens and because of that it is much easier to generate realistic looking images. However, it was never suitable for real-time applications in the past due to enormous amount of processing necessary.

Even though ray tracing is relatively new in the realm of real-time graphics, the concept of ray tracing is almost as old as rasterization. In 1968 Arthur Appel explores the idea of casting a ray through a pixel to determine, which surface is visible, and also the idea of casting a ray from a surface point to the light source to check, if the light reaches the surfaces unobstructed [App68]. Later in 1980 Turner Whitted introduces the well-known ray tracing model known as "Whitted-Raytracing" [Whi80]. If a surface is hit by a ray, additional rays are cast for reflection, refraction and shadow. In the 1990s ray tracing started to be used in visual effects for movies like "Terminator 2: Judgment Day" (1991) or "Jurassic Park" (1993). In the 2010s animation studios like Pixar move to fully path traced rendering [Chr+18], which uses ray tracing to trace the whole path from a light source to the camera in order to generate photorealistic images. An example for this is Pixar's animated movie "Finding Dory" (2016) [HVF16].

In 2018 Nvidia paved the way for ray tracing in real-time applications by introducing the RTX-Series. These graphics cards used the "Turing architecture", which introduced hardware-accelerated ray tracing with the new RT Cores. RT Cores traverse the BVH (Bounding Volume

Hierarchy) autonomously, and by accelerating traversal and ray/triangle intersection tests, they offload the SM (Streaming Multiprocessor), allowing it to handle other vertex, pixel, and compute shading work [Cor18].

Having to test every ray against every triangle for intersection is not feasible at all and takes way to much processing power, even for non-real-time applications. A much smarter way to find the triangles, that are hit by a ray, is using a bounding volume hierarchy (BVH). It is a tree structure of volumes (usually AABBs), which devide the scene into smaller and smaller subsections as we go down the tree. We can prune a branch as soon as we realize, that the section does not intersect with the ray and therefore avoid a lot of ray/triangle intersections.

In this thesis we will explore the topic of ray tracing in dynamic environments. We are going to look at objects, that move through a scene, but also at objects that deform their mesh using skeletal animation. Specifically, we are going to use the Vulkan API and look at how it implements ray tracing support. We are going to learn how to create the raytracing pipeline, setup our shader binding table and build our acceleration structures as well as the different ways we can update them later during an animation. We are then going to test how different scenes perform using different approaches and compare them to each other.

2. Related Work

2.1. Real-Time Ray Tracing on GPUs

2.2. Dynamic Scene Ray Tracing

2.3. Acceleration Structures and Memory Management

2.4. Summary and Research Gap

3. Technologies

3.1. System Information

- **Operating System:** Ubuntu 24.04.2 LTS
- **CPU:** Ubuntu 24.04.2 LTS
- **GPU:** Nvidia

3.2. Build Tools

- Premake 5
- GNU Make 4.3
- Visual Studio 2022

The project uses Premake 5, which does not build the project directly, but it is a command line utility which reads a scripted definition of a software project and, most commonly, uses it to generate project files for toolsets like Visual Studio, Xcode, or GNU Make [ric23]. It is a simpler alternative to CMake. On Linux we use it to create “Makefiles”, which then ultimately build our project. On windows we create Solution- and Project-files for Visual Studio 2022 and we can build and run the project inside Visual Studio.

3.3. Graphics API

As the graphics API for our project we chose **Vulkan 1.2**. Vulkan is a modern platform-agnostic API to handle GPU operations. It is the successor to OpenGL, which was discontinued in 2016. Vulkan allows for much more fine-grained control over the GPU, supports multi-threading and for this project most importantly raytracing, through extensions like `VK_KHR_ray_tracing_pipeline` and `VK_KHR_acceleration_structure`. Since these extensions were introduced in Vulkan 1.2, that is the version used in the project.

3.4. Programming Language

At the start of the project Rust was considered as the projects programming language. Rust is a new, memory-safe, low-level programming language and a modern alternative to languages

like C and C++. While Rust does have a lot of advantages over C++, we chose C++ in the end. The main reason for this is that the Vulkan API was designed for C/C++. There are a lot of great Vulkan-libraries for Rust, like the popular "Ash" create, which is generated from "vk.xml". However, way you use the API does differ from the classic C API. This could be problematic, considering that nearly all the few resources relating to raytracing in Vulkan use C/C++. This could have made it difficult to translate the raytracing code into Rust later down the line, so C++ was the safer choice.

3.5. File Formats

3.5.1. glTF

The dynamic 3D scenes, that the project uses, are stored in the **glTF-2.0** file format. It stores its models in a scene graph and supports skeletal animation as well as morph animations. By default, it stores the scene data in the popular JSON format. Additionally, buffers are stored in binary in BIN files and textures are stored in image files (PNG, JPEG, ...). Since the file format was developed by Khronos Group, which is also behind Vulkan, the data inside the buffers is laid out in a way that makes it easy to use in Vulkan.

3.5.2. CSV

3.6. Libraries

3.6.1. Volk

volk is a meta-loader for Vulkan. It allows you to dynamically load entry points required to use Vulkan without linking to vulkan-1.dll or statically linking Vulkan loader. Additionally, volk simplifies the use of Vulkan extensions by automatically loading all associated entry points [<https://github.com/zeux/volk>].

3.6.2. VMA (Vulkan Memory Allocator)

Memory Management in Vulkan is quite difficult. For example, when you create a buffer or image in Vulkan there is no memory associated with it. The programmer needs to manually allocate block of memory with a certain type from a certain memory heap. The properties of these memory heaps depend on the underlying physical device and need to be queried first. Additionally, creating a new memory allocation for each resource may be suboptimal, since the number of allocations can be very limited and many allocations may negatively impact performance. VMA (Vulkan Memory Allocator) simplifies, automates and optimizes memory management in Vulkan.

3.6.3. vk-bootstrap

Initializing Vulkan involves a lot of boilerplate code, which is going to be basically the same in every Vulkan application. VkBootstrap can be used to reduce the amount of boilerplate code in a project.

This library simplifies the tedious process of:

- Instance creation
- Physical Device selection
- Device creation
- Getting queues
- Swapchain creation

[Lun25]

3.6.4. cgltf

To load a glTF file in the project, cgltf is used. cgltf parses the glTF file, which is stored in JSON. It would also have been possible to read the JSON directly, but cgltf takes care of validating the file, loads the required buffers from their BIN-files into memory and creates a C-structure, which is a lot more comfortable to use. Our project does use not this structure directly, but rather used to generate a Scene object.

3.6.5. CLI11

CLI11 is a header-only library written in modern C++, that is designed to parse the command-line options of your application. It is simple, feature-rich and safe. CLI11 allows you to define flags, options, commands and subcommands. You can also add certain restrictions to your parameters, like only allowing positive numbers or that a parameter is an existing path.

4. Implementation

4.1. Project Setup

The projects as well as this thesis' source code are available in the following Github repository:

https://github.com/mulsicpp/bachelor_thesis_2025.git

The project is organized in the following directories:

- "assets": This folder contains the projects' assets. It is further divided into scenes, which contains the glTF test scenes, and "shaders", which contains the shaders, that are used in the different pipelines.
- "external": This folder contains all of the "complex" external libraries. "Complex" in this context means, that the library has multiple header or source files or it is in binary form.
- "premake": This folder contains the premake executables for both windows and linux.
- "src": This folder contains all of our source files for the project. It is described in more detail in the next chapter.

Additionally, there are some scripts, that are used to build the project.

4.1.1. Source Code Organization

In this section we will specifically look at how the "src" folder is organised.

Directly in the "src" folder we find "main.cpp", which contains our entry point of the application. The "external" folder once again contains external libraries, but this time the libraries, which are in the form of a single header file. These usually need to be included in a C/C++ file and need to be compiled. That's why it makes sense to keep these libraries with our other source files. The "utils" folder contains utilities, that may be useful all throughout the project. It includes functionalities like debug logging, move semantics and file loading. All the folders with a "vk_" prefix contain code, which abstracts the low-level Vulkan API code. The goal of these abstractions is, that the low-level code should only ever be used in these folders. The rest of the project should only use the abstracted API. Each of the folders focuses on a different area of Vulkan:

- "vk_core": The most important class in this folder is the Context class. It is designed as a singleton, which contains the VkInstance, VkPhysicalDevice, VkDevice, VkQueues and VmaAllocator. After the context gets created it can be accessed anywhere. The

folder also contains the `Handle<T>` class, which ensures that a handle only has one owner, the `CommandBuffer` abstraction and functionality to deal with `VkFormat`.

- "vk_resources": This folder contains Vulkan resource objects like `Buffer` and `Image`, as well as `ImageView` and `SubBuffer`.
- "vk_pipeline": This folder contains Vulkan objects, that are related to the setup of a pipeline. This includes `Pipeline`, `PipelineLayout`, `Shader`, `RenderPass`, `Framebuffer` and `DescriptorPool`.
- "vk_sync": Contains the Vulkan synchronization objects `Fence` and `Semaphore`.
- "scene": This folder contains the `Scene` class, which describes a dynamic scene with a scene graph. All the other classes used by or contained in the scene, like `Mesh`, `Node`, `Camera` and `Animation`, can also be found in this folder.
- "rendering": This folder contains 3 different renderers:
 - `Rasterizer`: Render a scene from the viewpoint of a camera into a framebuffer using traditional rasterisation.
 - `Raytracer`: Render a scene from the viewpoint of a camera into an image using modern hardware-accelerated raytracing.
 - `Skinner`: Calculates the skinned positions of a mesh during an animation.

4.2. Vulkan Abstraction

4.2.1. Move Semantics

As already mentioned in the `Methods and Technology` chapter, Rust was originally considered as programming language, because it is modern and memory safe. One of Rust's core features is ownership, which means that each value/object can only be owned and managed by only one variable, and when that variable goes out of scope, it alone is responsible for cleaning up the object. When that variable gets assigned to another variable, the new variable becomes the new owner (i.e. the value gets moved). Ownership would be a very nice feature to manage Vulkan objects. Vulkan objects should not be copied on assignment but only moved and when the object is no longer needed (i.e. the owner goes out of scope), it should automatically be destroyed. While C++ does not support ownership directly, it does have so-called move semantics, with which we can emulate ownership. The ownership of Vulkan handles is managed by the `Handle<T>` class. To do this the class declares a default constructor, move constructor and destructor and overrides the move assignment operator. It is also important, that the copy constructor and copy assignment operator are deleted. The reason for that is that copying a Vulkan object is usually very expensive or not even possible and it is almost never necessary. When we want to copy an object, we need to explicitly create a new one first. This avoids unnecessary and complicated accidental creation and copying of new objects.

4.2.2. Shared Pointers

Since C++11 the language supports so-called smart pointers, which make memory management safer and less complicated. One of these smart pointers is `std::shared_ptr<T>` (or our alias `ptr::Shared<T>`). This type of pointer allocates an object, which can then be shared between multiple owners. The object lives while it has at least one owner, and the last object to go out of scope cleans up the object. This structure can be combined very nicely with our previously defined handle. A Vulkan object can only have one owner, which is a harsh limitation and often causes problems. If we wrap the object in a shared pointer, it can be shared across multiple owners. We also added the class `ToShared<T>`, which a class can inherit from. An instance of that class can then be moved into a shared pointer by calling `to_shared()`.

4.2.3. The Builder Pattern

As mentioned, before we want the creation of Vulkan objects to be very explicit. The first method to create objects in C++ you think of are constructors, but they prove to be problematic for a couple of reasons. The first problem is that constructors are often called implicitly (e.g. the default constructor gets called when declaring a variable). Default construction only makes sense for very few Vulkan objects (like for example a `VkFence`) and we do not want a Vulkan object to be created when implicitly (or explicitly) calling the default constructor. So, for that reason, the default constructor simply does not create a Vulkan object or, in other words, leaves the handle at `VK_NULL_HANDLE`. Now if a different constructor actually creates an object, the behavior of constructors becomes inconsistent regarding Vulkan object creation. In addition to that, Vulkan objects are almost always created using a `*CreateInfo` struct, which may take a lot of parameters. To solve all of these issues we implemented a `Builder` class for most Vulkan objects. Some exceptions are `Fence` and `ImageView`, which are created with `Fence::create` and `ImageView::create_from` respectively. This makes it very clear when a Vulkan object is created, since it can only be created using a `Builder` or a static method (like `Fence::create`) and especially it can never be created by a constructor. This is also the reason Vulkan objects never have constructor, besides the default constructor.

```
vk::Buffer buffer; // declare a buffer (handle is VK_NULL_HANDLE)

buffer = vk::BufferBuilder()
    .add_usage(VK_BUFFER_USAGE_TRANSFER_DST_BIT)
    .add_usage(VK_BUFFER_USAGE_VERTEX_BUFFER_BIT)
    .memory_usage(VMA_MEMORY_USAGE_GPU_ONLY)
    .size(128)
    .build();
```


4.2.4. Basic Vulkan Object Structure

All of the Vulkan objects follow a basic structure, which can be seen in the following code sample:

```
class Foo : public utils::Move, public ptr::ToShared<Foo> {
    friend class FooBuilder;
private:
    Handle<VkFoo> foo_handle{};
    // Additional member variables

public:
    Foo() = default;

    inline VkFoo handle() const { return foo_handle; }
    // Additional getter functions

    // Additional functions
};

class FooBuilder {
public:
    using Ref = FooBuilder&;
private:
    // build parameters, e.g. VkFormat _format;

public:
    FooBuilder() = default;

    // parameter setter functions, e.g. Ref format(VkFormat format);

    Foo build() const;
}
```

Figure 4.1.: An example of how to use command buffers in your code.

It is important to note, that `Foo` is just a placeholder for an actual Vulkan object (like `Buffer`, `Image` or `Pipeline`). Usually, the name of the class is just the name of the Vulkan handle without the "Vk" prefix (e.g. `VkBuffer` $\implies$ `Buffer`), but there are some exceptions. For example a `Shader` class holds a `VkShaderModule` handle. As we can see, the Vulkan handle is wrapped in the previously mentioned `Handle<T>` class, which ensures correct move semantics. The member function `handle` allows us to access the raw Vulkan handle.

The class inherits from `utils::Move` and `ptr::ToShared<Foo>`. The `utils::Move` class implements the default move constructor and move assignment, and it deletes the copy constructor and copy assignment. The `ptr::ToShared<Foo>` class contains the function `to_shared`, which is just syntax sugar for moving the object into a shared pointer.

The additional variables in the class are often used to store meta-data (e.g. the format and extent of an image) or to keep other objects alive in a shared pointer, that are referenced by the underlying object (e.g. the render pass of a pipeline). There usually is also an getter function for these variables.

The additional functions include functions, that actually use or manipulate the underlying object. Often, these functions have a "cmd_" prefix, which means that they are wrappers around one or multiple commands (e.g. `Buffer::cmd_copy_into`).

Even though not every Vulkan object has uses a builder class, it is also shown in the sample code, since most Vulkan objects use it.

4.2.5. The Vulkan Context

Before we can actually do any work in Vulkan, we need to do a lot of setup. This usually includes the following steps:

1. Create a `VkInstance`.
2. Select a `VkPhysicalDevice`, which supports the extensions, features and capabilities required by the project.
3. Create a `VkDevice` from the selected `VkPhysicalDevice`.
4. When creating the `VkDevice`, `VkQueue`s from a certain queue family need to be enabled, to be able to submit command buffers later.

These Vulkan objects are accessed very frequently throughout the project. For example, you need to pass the `VkDevice` as a parameter when creating or destroying most Vulkan objects and a command buffer, which executes a sequence of Vulkan commands, needs to be submitted to a certain `VkQueue`. Keeping track of all of these Vulkan objects is quite tedious, which is why we introduced the Vulkan context. It is represented by the `Context` class, which stores these Vulkan objects, allows the programmer to access them anywhere and reduces the effort of setting up Vulkan.

The Vulkan context contains the following objects:

- the `VkInstance`
- the `VkPhysicalDevice` and `VkDevice`
- an optional `GLFWwindow*` from which a `VkSurfaceKHR` is created and also stored (this is necessary for rendering to a window, but it will not be used in our main application, since it is headless)

- the `CommandManager`
- the `VmaAllocator`

The class is designed similar to a singleton, but it deviates in some parts. A singleton usually creates its instance at the start of the application or when it is used for the first time (lazy instantiation). However, `Context` can not be implicitly instantiated this way, because it needs some parameters stored in a `ContextInfo`, when being instantiated. For this reason we need to explicitly create the Vulkan context using `Context::create(...)` with the desired `ContextInfo` before calling `Context::instance()` for the first time or we will get a `nullptr`. While all other objects in our project get cleaned up automatically when going out of scope, the Vulkan context also needs to be destroyed explicitly using `Context::destroy()` when it is no longer needed. If we now want to get the current `VkDevice`, we can call `Context::instance()->get_device()`.

4.2.6. The Command Manager

Queue management in Vulkan is a big and complex topic. Vulkan exposes the different queue families on your GPU, which support a subset of the following command types:

- Graphics
- Transfer
- Compute
- Present

When submitting a command buffer to a queue, the queue needs to belong to a queue family, which supports the different types of the commands in the command buffer.

The management of Vulkan queues is completely up to the programmer. You can query the capabilities of the queue families and then enable an arbitrary amount of queues from different queue families. This is especially useful in a multi-threaded application, where you can submit two command buffers to two different queues and they will be executed in parallel.

The `CommandManager` class makes queue management easier. It manages the queue families, as well as their enabled queues and created descriptor pools. Our application is single-threaded in order to avoid too much complexity and because our main objective is to benchmark the performance of dynamic raytracing, which works better in a single-threaded environment. For this reason we also do not need multiple queues or queues designated for a certain type of operations (for example a designated transfer queue). However we do need a queue for every command type, which our command manager ensures. For every command type, it will iterate over all queue families and select the first family, which supports the command type. When creating the device, we enable the first queue of every queue family, which has been selected at least one time for a command type.

The following are some examples of possible queue setups:

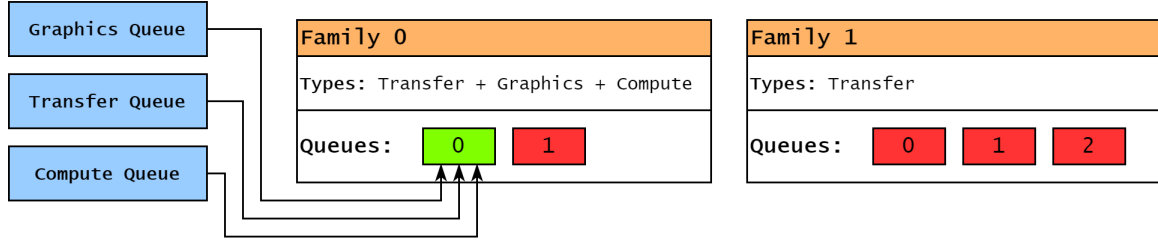


Figure 4.2.: Queue setup example with only one queue

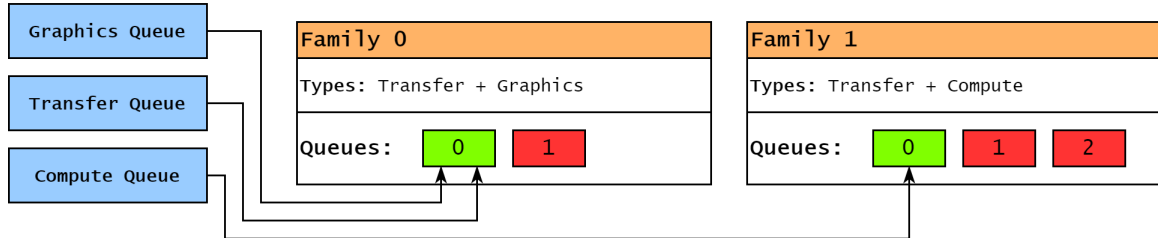


Figure 4.3.: Queue setup example with two queues

We also need a command pool to be able to allocate command buffers. When creating a command pool, a queue family has to be specified. Because the allocated command buffers can only be submitted to queues of the specified family, we create one command pool for every queue family that is used.

4.2.7. The VMA Allocator

As already mentioned in the chapter Technologies (subsection 3.6.2), our project uses VMA (Vulkan Memory Allocator) to make memory management in Vulkan easier. In order to be able to allocate memory for our buffers and images later, we need an allocator. The allocator should be easily accessible when we create a buffer or image and for that reason we will also create and store the VMA allocator in the Vulkan context. We will use this allocator later, when allocating our resource objects (see subsection 4.2.9).

4.2.8. Command Buffers

As we know, commands can not be executed in Vulkan directly, like they can for example in OpenGL. Instead, you first need to allocate a command buffer from a command pool. You can then record all of the commands you want to execute into the buffer and can then submit it for execution to a queue, that supports the type of the commands. It is important to note

that when you submit the command buffer the current CPU-thread does not wait for all the commands to finish. This allows for the CPU to keep working, while the GPU runs our commands. If we want to wait for the command buffer to finish, we can specify a fence when submitting the command buffer, which will be signaled once all commands have finished.

We abstract this logic in the `CommandBuffer` class. It contains a `VkCommandBuffer` handle, a `Queue`, which tells the command buffer, where it should be submitted to, and a `Fence` (see subsection 4.2.10), which is used to wait for the command buffer to finish executing.

The `CommandBufferBuilder` has the following two parameters:

- `queue_type`: A value of type `QueueType`, which contains the elements `Transfer`, `Graphics`, `Compute` and `Present`. It is a mandatory parameter, which tells the builder to which of the queues inside the queue manager the command buffer is going to be submitted to (for example `QueueType::Graphics` means the command buffer will be submitted to the graphics queue of the command manager).
- `single_use`: An optional boolean value, which is set to `false` by default. If set to `true`, it sets the `VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT` flag when recording the command buffer. This flag specifies, that the command buffer will only be submitted once, which lets the driver apply optimizations.

Inside `CommandBuffer` we can find the following member functions:

- `Ref record(CommandRecorder)`:

This function takes in a `CommandRecorder` as argument, which is an alias for `std::function<void(ReadyCommandBuffer)>`. In other words it is a function pointer or lambda function, which takes a `ReadyCommandBuffer` as an argument and returns `void`. A `ReadyCommandBuffer` is simply a wrapper around a `VkCommandBuffer` handle, which can only be constructed by the command buffer. The job of the `CommandRecorder` is, to record commands into the command buffer.

Functions, that record a commands can be recognized by their `"cmd_"` prefix. These functions have to be called inside a `CommandRecorder`, because thier first argument is always a `ReadyCommandBuffer`.

- `Ref submit(SubmitInfo)`:

This function submits the command buffer to the queue, for which it was created for. The `SubmitInfo` parameter is optional and empty by default. This parameter allows us to specify semaphores, that we want to wait for before executing the command buffer, and semaphores, that we want to signal once the command buffer is finished executing (see subsection 4.2.10).

- `Ref wait()`:

This functions simply waits for the execution of the command buffer to finish using a fence. If the command buffer is not currently getting executed, the function does nothing.

It is also important to mention, that this function also gets called in the destructor of the `CommandBuffer`, since it is not allowed to destroy a `VkCommandBuffer` while it is in execution. This also means, that a command buffer implicitly wait for execution to finish, when it goes out of scope.

The underlying fence uses the flag `VK_FENCE_CREATE_SIGNALED_BIT`. Now if we call wait on a newly created command buffer, it will not block the thread, since the fence is signaled. We only reset the fence right before submitting the buffer for execution, because we only want to wait for the buffer while it is in execution.

The `CommandBuffer` class also contains the following static method:

```
static void single_time_submit(QueueType, CommandRecorder)
```

It creates a single-use command buffer for the queue with the specified `QueueType`. Then, it records the given `CommandRecorder`, submits it with an empty `SubmitInfo` and waits for execution to finish. It is basically just a quality-of-life function for creating and executing single-use command buffers.

The following code snippet shows, how the command buffer and its functions can be used:

```
// create the command buffer
vk::CommandBuffer cmd_buf = vk::CommandBufferBuilder()
    .queue_type(QueueType::Graphics)
    .single_use(true)
    .build();

// create a callback, which records the commands
vk::CommandRecorder recorder = [&] (ReadyCommandBuffer ready_buf) {
    // record commands
    // ...
}

// record the commands, submit to queue and
// wait for execution to finish in one line
cmd_buf.record(recorder).submit().wait();
```

Figure 4.4.: An example of how to use command buffers in your code.

4.2.9. Resource Objects

Resource objects are used to store data on the GPU, which can then be read and written to in shaders. In Vulkan we distinguish two types of resource objects:

- **Buffer:**

Buffers represent linear arrays of data which are used for various purposes by binding them to a graphics or compute pipeline via descriptor sets or certain commands, or by directly specifying them as parameters to certain commands [Khr25a]. The data is raw and not associated with any format, layout or dimensionality, therefore very flexible.

Common usages:

- Vertex buffer
- Index buffer
- Uniform buffer
- Storage buffer

- **Image:**

Images are specialized resources that have multi-dimensional access rather than the typical linear access to memory [Khr25b]. They can also be multi-dimensional and are associated with a format and layout.

Common usages:

- Texture
- Render target

We represent these two objects by the `Buffer` class and `Image` class. Since they are both resource objects, they are very similar in certain aspects. One challenge, that they both share, is memory management, which can be quite difficult.

When creating a resource object, they are not actually backed by memory immediately, but you need to allocate a block of device memory and bind that memory to the resource. When allocating the memory, you need to specify a memory type, which corresponds to a certain memory heap and can have a set of properties, which impact the visibility of your memory as well as performance. It is also important to mention, that too many allocations are not just bad for the performance of your application, but there is also a limit of allocations, that cannot be superseded.

To make our lives a little bit easier, we use VMA (Vulkan Memory Allocator) (see subsection 3.6.2).

The most important parameter we need, when using VMA is the `VmaMemoryUsage`, which describes the properties of the memory allocation. We use three of them in our project:

- `VMA_MEMORY_USAGE_CPU_ONLY`:
- `VMA_MEMORY_USAGE_GPU_ONLY`:
- `VMA_MEMORY_USAGE_CPU_TO_GPU`:

4.2.10. Synchronization Objects

A lot of vulkan operations can be executed in parallel, which is why we need **synchronization objects**. There are two types:

- **Semaphore:**

A semaphore is used to synchronize two operations, which both happen on the GPU. We do this by specifying an array of wait-semaphores and an array of signal-semaphores. The GPU operation waits for all wait-semaphores to be signaled before starting and signals all signal-semaphores when it is finished.

- **Fence:**

A fence is used to synchronize operations on the GPU with the CPU. This is done by specifying a fence when starting a GPU operation. When the GPU operation finishes, the fence gets signaled. The CPU can wait on a fence, which will block the thread until the fence gets signaled.

The `Semaphore` class encapsulates a semaphore. It does not have any other functionality, other than holding a `VkSemaphore` handle (subsection 4.2.4), because it does not need it. A semaphore only ever gets passed as a parameter to GPU operations and these GPU operations signal and unsignal the semaphore. You never use the semaphore explicitly, but it is used implicitly by GPU operations. The `VkSemaphoreCreateInfo` does not have any member variables (other than `sType` and `pNext`). Therefore a builder would be a very clumsy way to create a semaphore. Instead we use the parameterless, static method `Semaphore::create`.

The `Fence` class encapsulates a fence. While still being a very simple class, it does have a bit more functionality than `Semaphore`. A fence has two important member functions:

- `void reset()`:

This function resets the fences state to "unsigaled". A semaphore automatically gets set to unsigaled by the GPU operation that waits for it. Once a fence has been signaled, it will stay signaled until we manually reset it.

- `void wait(uint64_t timeout)`:

This function will block the current thread until the fence gets signaled or `timeout` nanoseconds pass. The `timeout` parameter is optional and set to `UINT64_MAX`, which essentially makes the function indefinitely.

Eventhough `VkFenceCreateInfo` has one member variable, we use the static method `Fence::create` instead of a builder to create a fence. This method has the argument of type `VkFenceCreateFlags`. The `VkFenceCreateFlags` enum only contains the flag `VK_FENCE_CREATE_SIGNALED_BIT`. When setting this flag the fence will be created in the "signaled" state, otherwise the fence will be created in the "unsigaled" state. This is useful for our command buffer implementation (see subsection 4.2.8).

4.2.11. Pipeline-related Objects

In this section we will talk about all the different Vulkan objects that are related to creating and using a pipeline. We will focus on the classical rasterization pipeline, which we also use for skinning in our application, but a lot of these objects are also used in the raytracing pipeline (see subsection 4.2.12).

Shaders

A `Shader` stores the code of a shader in Spir-V as well as the shader stage, that it is used in. The code can either be given directly in binary or be loaded from a specified file.

Vertex Input

The `VertexInput` struct describes the data, that gets passed to the vertex buffer. It has an array of `VertexBindingInfos` and an array of `VertexAttributeInfos`. Every binding info has a unique `binding` index, a `stride`, which is the distance between two elements in bytes, and an `input_rate`, which specifies if an element should be read for every vertex or for every instance. Each attribute info is part of a specific `binding` (referenced by the binding index) and has a unique `location` index, an `offset` and a `format`. In order to access an attribute in the vertex shader, we need to specify its location (e.g. `layout(location = 0) in vec3 positions;`).

Render Pass

A `Renderpass` stores a set of `Attachment`s, that the shaders in a pipeline can output to. Each of these attachments holds information about their type (e.g. color, depth, stencil), format as well as load and store operations. Note that our implementation does not support stencil attachments, since they are never used in our project. Our appciation also does not allow sub-passes, or rather exactly one sub-pass per render pass.

Framebuffer

Just like an object is an instance of a class, a framebuffer can be seen as an instance of a render pass. It holds a set of image views, that are used as render targets. Each image views format and aspect needs to match their respective attachment in the render pass.

Pipeline

A pipeline describes the whole rendering process. It contains the render pass and the different shaders, vertex input and pipeline layout, as well as information for:

- input topology
- rasterization

- depth testing
- color blending
- dynamic states

Most of this information is hard coded, since we don't need to change it in our application (depth testing, color blending, dynamic states). We can however change the input topology and disable rasterization (later used for GPU skinning).

Pipeline layout and descriptor pool

In order to be able to use data in a shader, that is not vertex data (e.g. uniform buffers, storage buffers, samplers, acceleration structures, ...), we need to describe the data using descriptor sets. These descriptor

4.2.12. Raytracing-related Objects

4.3. Scene

In order to understand, why we structured our scenes the way we did, we first need to take a look at the glTF file format, that stores our scenes. As already mentioned (see subsection 3.5.1).

4.3.1. glTF Structure

A glTF stores its data in the JSON format. There are different types of objects in glTF, each of them is stored in an array. An object often references another object by its index in the respective array. The objects we are interested in are:

- **Buffer, Buffer view and Accessor**

These three objects work together closely. A buffer stores a block of binary, usually contained in a separate BIN file. A buffer view describes a certain part of a buffer. It references a certain buffer and contains an offset, length and optional stride. A buffer view sees the data as just raw data. To associate some kind of meaning to the data we use an accessor. It references a buffer view with a certain offset and also contains the number of elements, as well as the type (e.g. scalar, vec3, mat4, ...) and component type (e.g. float, unsigned int, ...). Accessors are later used for things like vertex attributes and joint matrices.

- **Mesh**

A mesh stores a static 3D model. It contains a material and an array of primitives. Each primitive has a mode (e.g. points, lines, triangles), and optional accessor for indices and a set of accessors used for the different vertex attributes (e.g. positions, normals, uv coordinates, vertex colors, joint weights).

- **Skin**

A skin is used for skeletal animation of a mesh. It contains an array of nodes, that are used as the skeletons joints, and an accessor for the inverse bind matrices of the joints.

- **Node**

This represents a node in the scene graph. A node always has a transform, which is either represented by a 4x4 matrix or a scale(vec3), rotation(Quaternion) and translation(vec3). A node can have an array of child nodes, which inherit the transform of the parent node. In other words, a nodes transform is applied in the parent nodes coordinate system.

A node can optionally also contain a mesh, which is gonna be transformed by the nodes transform. If a mesh is present, the can also contain a skin. The nodes skin will deform the nodes mesh during an animation.

- **Scene**

A scene contains an array of root nodes. These nodes as well as all of their children are part of that scene.

- **Animation**

An animation has an array of channels and an array of samplers. A channel describes, which property is being sampled during the animation. It contains the effected node and a path the effected property of the node (e.g. "transform", "scale", "rotation"). It also holds a reference to one of the animations samplers.

A sampler is used to calculate the value of a property at a certain point in time from a set of key frames. It contains two accessors, one for the time of the key frames and one for the value of the keyframes. Additionally, a sampler contains the type of interpolation, that should be used to sample a value.

There are also some other objects (e.g. materials, textures and cameras), but since we dont use them for our project, we don't explain them here.

4.3.2. Scene Structure

We will now take a look at the structure of our scene implementation. For this we will look at the different C++ classes we used.

The `Scene` class

The `Scene` class represents a scene that can be loaded from a glTF file. It contains an array of nodes, an array of animations and an array of skins. Additionally, it stores the TLAS, that is used for the ray traversal through the scene, and the different buffers used by the scene in a `SceneBuffers` struct.

A scene can be loaded using the static method `Scene::load`, with the path to the glTF file as parameter. A scene also contains the member function `Scene::iter`, which returns

a node iterator for all of the nodes contained in the scene. There also is the function `Scene::build_acceleration_structures`, which we need to call on a scene to build all of the necessary acceleration structures so we can use ray tracing. The rest of the scenes functions are explained in the next subsection about updating a scene (see subsection 4.3.3).

The `Node` class

The `Node` class directly represents a node in glTF. As we mentioned previously the transform of a node can either be stored as a matrix or separated into a translation, rotation and scale. To model that in our application, we added the struct `RawTransform`, which contains the transform, rotation and scale. We also have the struct `NodeTransform`, which stores the local transform of a node and looks like this:

```
struct NodeTransform {  
    bool raw;  
    union {  
        glm::mat4 matrix;  
        RawTransform raw;  
    } transform;  
}
```

The union `transform` is used to store either a `glm::mat4` or a `RawTransform`. The boolean `raw` is used to determine, how the union should be interpreted, i.e. if `raw` is `true`, the union contains a raw transform, otherwise it contains a matrix. The node also stores the global transform as a 4x4 matrix, but it is only valid after calling `Node::update_global_transform`.

A node also stores an array of child nodes and it can also contain a mesh. If a mesh is present, it also stores a BLAS and the node must generate a TLAS instance that references the nodes BLAS when building the acceleration structure. After building the acceleration structure for the first time, the node also stores the index of the TLAS instance, which is necessary when updating the TLAS instance later. If the node does not have a skin, the node's BLAS is the the same as the one inside the mesh.

If the node does however contain a skin, it stores an array of sub-buffers for dynamic positions. For every primitive of the node's mesh, a sub-buffer for dynamic positions needs to be present in the array. Each sub-buffer needs to have the same size as the sub-buffer for the static positions of the respective primitive. These dynamic positions are used to store the vertex positions of a mesh after skinning. If a skin is present, the BLAS is not simply just a reference the meshes' BLAS, but it is a separate, dynamic BLAS, which can be updated.

The `Mesh` class

The `Mesh` class represents a glTF mesh. It stores an array of `Primitives`. Each primitive stores three sub-buffers: `positions` for the vertex positions, `indices` for the vertex indices and `joint_weights` for the joint weights used for skinning the mesh. The positions and

indices are pretty straight forward and just store a sequence of `vec3s` and `unsigned ints` respectively. The joint weights are a bit more complex, because they are stored in a 2D array of `JointWeight` structs. A `JointWeight` contains the index of the node in the skeleton of the skin and its weight. All the arrays of joint weights for a vertex are stored in sequence like shown in this figure:

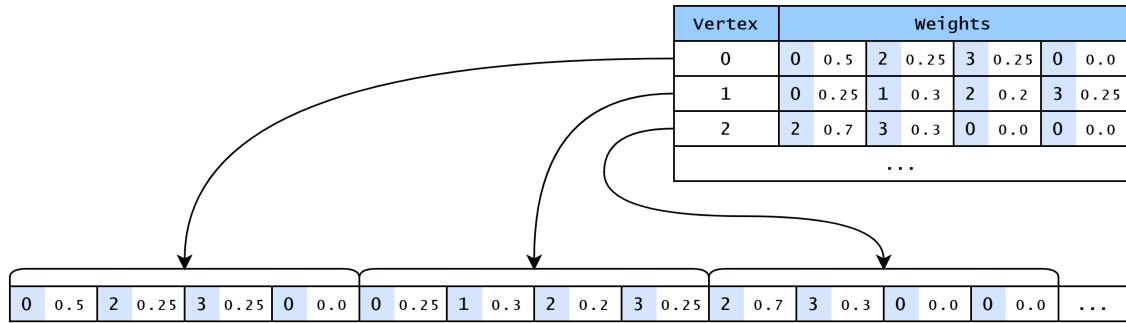


Figure 4.5.: Joint weight layout

Additionally, each primitive also contains the member variables `positions_cpu` and `joint_weights_cpu`, which store the same data as `positions` and `joint_weights`, but this time inside `std::vector`s for easier access in CPU skinning.

Finally, a mesh can also contain a BLAS, which is created for the static mesh. It is only created if a mesh is used statically by a node, i.e. a node that uses the mesh without a skin.

The `Skin` class

The `Skin` class is relatively simple and matches the glTF skin object almost exactly. It simply stores an array of `Nodes`, which represent the joints in the skeleton and an array for the inverse bind matrices, with one entry for each joint.

The `Animation` class

The `Animation` class represents a glTF animation. It contains an array of `AnimationSamplers`, which contain a reference to the node, that they modify during animation, and also the virtual function `AnimationSampler::apply_for`. This function samples a value at the specified point in time and applies it to a certain property of the nodes transform. There are three sub-classes of `AnimationSampler`, which override the `apply_for` function:

- `TranslationSampler`: samples a 3D vector and sets it as the nodes translation.
- `RotationSampler`: samples a quaternion and sets it as the nodes rotation.
- `ScaleSampler`: samples a 3D vector and sets it as the nodes scale.

The animation itself also has an `apply_for` function, which iterates over all animation channels and applies them for the given point in time.

Buffers and Sub-Buffers

glTF uses buffers, buffer views and accessors. We don't use the data in the glTF buffers directly, but we read the data and lay it out in a more convenient way. A glTF accessor may access data, that is interleaved with other data, or sparse data. Additionally, the accessor's component type may not match our component type. However, we want to have our data lined up in memory sequentially and of a fixed type. Luckily, the `cglTF` library offers the function `cglTF_accessor_unpack_floats`. This function reads all the values of the accessor, converts the components of the values to `float` and stores the values sequentially in a target buffer. The function `cglTF_accessor_unpack_indices` does the equivalent for `unsigned ints`.

We create one buffer for each usage type of our data. We have one buffer for the vertex indices, one for static vertex positions of the mesh and one for dynamic vertex positions, that may be different for each mesh. An attribute can then reference a sub-subsection into one of these buffers using a sub-buffer. A sub-buffer has a reference to the buffer of interest using a `ptr::Shared<vk::Buffer>`, as well as an offset and a length. We do not need to store the type of the elements in the sub-buffer, since the type is fixed and can be determined by the variable. We know that the sub-buffers `positions` in `Mesh` and `dynamic_positions` in `Node` store a sequence of `vec3s`, while the sub-buffer `indices` in `Mesh` stores a sequence of `unsigned ints`.

The following figure gives a little bit more insight, of how these sub-buffers are used.

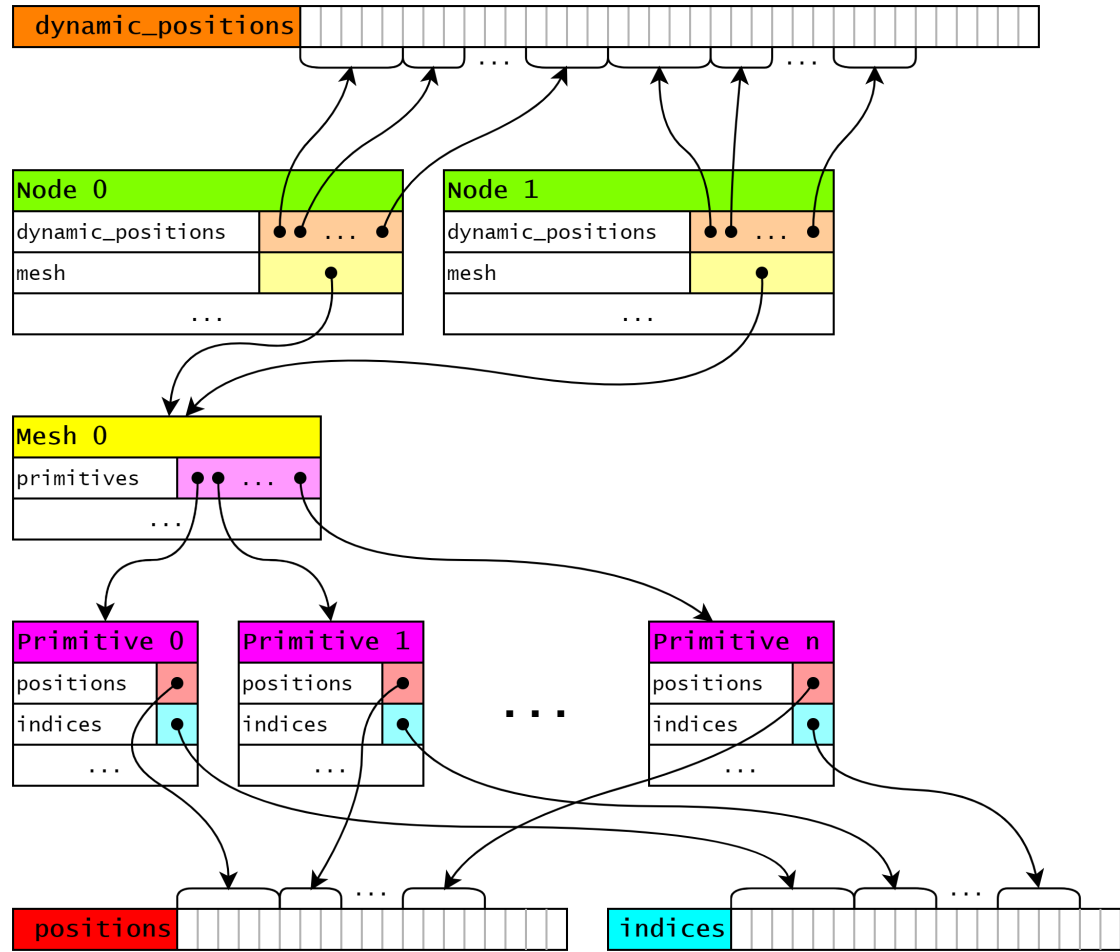


Figure 4.6.: Usage of sub-buffers in a scene

In this graphic we can see, how both nodes **Node 0** and **Node 1** reference the same mesh **Mesh 0**. This means, that they use the same sets of static positions and indices. But because two nodes with the same mesh can use different skins, each node needs to store their own set of dynamic positions. We can also see how the size of every **dynamic_positions** sub-buffer matches the size of the respective sub-buffer in **positions**.

4.3.3. Updating a Scene

Since our scenes are dynamic, we need a way to update them. We need update our scene after calling `Animation::apply_for` from an animation in our scene. A scene is updated in three steps:

1. updating the global transform for every node

2. skinning all animated models
3. updating the acceleration structure

When we want to update our scene we need to perform all of the above steps in the exact order. We keep the steps separate however, because we can use different implementations for some steps. For example, we can skin our models on the CPU or on the GPU and we can update our acceleration structure by either rebuilding it or refitting it.

Updating Global Transform

Using the member function `Node::update_global_transform` we can update the global transform of a node. This function takes in a `glm::mat4` as argument, which represents the parents transform. This argument is optional and gets set to the identity matrix by default. The following pseudo code shows the functions behaviour:

Algorithm 1 Pseudo code for updating the global transform

```
1: function UPDATE_GLOBAL_TRANSFORM(node, parent_transform)
2:   node.global_transform  $\leftarrow$  parent_transform  $\times$  node.transform
3:   for all child in node.children do
4:     update_global_transform(child, global_transform)
5:   end for
6: end function
```

The global transform is calculated by pre-multiplying the parents transform with the local transform of the node. We then call `Node::update_global_transform` recursively for every child node, with the new global transform of the current node as a parameter.

Skinning

Skinning describes the process of applying the deformation of a skeleton to a certain model. To do that we first need to calculate the joint matrices:

Algorithm 2 Calculating joint matrices

```
1: for  $i \leftarrow 0$  to joints.count - 1 do
2:   joint_matrices[i] = inverse(node.global_transform)
    $\times$  joints[i].global_transform
    $\times$  inverse_bind_matrices[i]
3: end for
```

After having calculated the joint matrices, we can skin the model:

Algorithm 3 Skinning a model

```

1: for  $j \leftarrow 0$  to (positions.count - 1) do
2:   matrix  $\leftarrow 0_{4 \times 4}$ 
3:   for all joint_weight in joint_weights[i] do
4:     matrix  $\leftarrow$  matrix + joint_weight.weight  $\times$  joint_matrices[joint_weight.index]
5:   end for
6:   position  $\leftarrow$  matrix  $\times$  position
7:   dynamic_positions[i]  $\leftarrow$  matrix  $\times$  positions[i]
8: end for

```

This algorithm uses **Linear Blend Skinning**, which is the standard skinning technique to its simplicity and efficiency [Jac+12]. We basically create a weighted sum of all the joint matrices used by the certain vertex and then multiply the resulting matrix with the vertex position to get the skinned position.

We have two implementations of the above skinning algorithm:

- **CPU skinning**

This implementation read all the vertex positions from the sub-buffer, runs the algorithm for each of them and stores the results in the respective sub-buffer for dynamic positions.

- **GPU skinning**

This implementation uses a vertex shader to run the algorithm for every vertex position and stores the resulting dynamic positions in a storage buffer. The pipeline that contains the vertex shader also disables rasterization, because no fragments should be generated during drawing and the pipeline should stop after running the vertex shader.

We could have also used a compute shader, which may perform better, but then we would need to implement a compute pipeline. With the approach we chose, we can simply repurpose the rasterization pipeline, which was already implemented.

Updating the acceleration structure

Algorithm 4 Updating acceleration structure

```
1: function UPDATE_ACCELERATION_STRUCTURE(scene)
2:   node_iterator = get_node_iterator(scene)
3:   tlas_instances = get_instances(scene.tlas)
4:   for all node in node_iterator do
5:     if has_blas(node) then
6:       if is_dynamic(node.blas) then
7:         update(node.blas)
8:       end if
9:       update_tlas_instance(tlas, node)
10:    end if
11:  end for
12:  update(scene.tlas)
13: end function
```

There is no `Scene::update_acceleration_structures` function, that you can call to update your acceleration structures. Instead, there are two slightly different member functions you can choose:

- `Scene::rebuild_acceleration_structures`
- `Scene::refit_acceleration_structures`

They both run more or less the same algorithm, the only difference is, that one uses "rebuilding" to update its acceleration structures and the other one uses "refitting".

5. Results

6. Discussion

7. Conclusion

A. General Addenda

If there are several additions you want to add, but they do not fit into the thesis itself, they belong here.

A.1. Detailed Addition

Even sections are possible, but usually only used for several elements in, e.g. tables, images, etc.

B. Figures

B.1. Example 1

✓

B.2. Example 2

✗

List of Figures

4.1.	An example of how to use command buffers in your code.	10
4.2.	Queue setup example with only one queue	13
4.3.	Queue setup example with two queues	13
4.4.	An example of how to use command buffers in your code.	15
4.5.	Joint weight layout	22
4.6.	Usage of sub-buffers in a scene	24

List of Tables

Bibliography

- [App68] A. Appel. “Some techniques for shading machine renderings of solids”. In: *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference*. AFIPS ’68 (Spring). Atlantic City, New Jersey: Association for Computing Machinery, 1968, pp. 37–45. ISBN: 9781450378970. DOI: 10.1145/1468075.1468082. URL: <https://doi.org/10.1145/1468075.1468082>.
- [Chr+18] P. Christensen, J. Fong, J. Shade, W. Wooten, B. Schubert, A. Kensler, S. Friedman, C. Kilpatrick, C. Ramshaw, M. Bannister, B. Rayner, J. Brouillat, and M. Liani. “An Advanced Path Tracing Architecture for Movie Rendering”. In: *ACM Transactions on Graphics*. Vol. 37. 3. 2018. URL: <https://docslib.org/doc/639313/an-advanced-path-tracing-architecture-for-movie-rendering>.
- [Cor18] N. Corporation. *Turing Architecture White Paper*. Tech. rep. NVIDIA Corporation, 2018.
- [HVV16] C. Hery, R. Villemin, and F. Hecht. *Towards Bidirectional Path Tracing at Pixar*. Tech. rep. 2016. URL: <https://graphics.pixar.com/library/BiDir/paper.pdf>.
- [Jac+12] A. Jacobson, I. Baran, L. Kavan, J. Popović, and O. Sorkine. “Fast Automatic Skinning Transformations”. In: *ACM Transactions on Graphics (proceedings of ACM SIGGRAPH)* 31.4 (2012), 77:1–77:10.
- [Khr25a] Khronos Group. *Vulkan® 1.4.327 - A Specification (with all registered Vulkan extensions)*. Accessed: 2025-09-19. The Khronos Group Inc. 2025. Chap. 12 Resources. URL: <https://registry.khronos.org/vulkan/specs/latest/html/vkspec.html#resources>.
- [Khr25b] Khronos Group. *Vulkan® 1.4.327 - A Specification (with all registered Vulkan extensions)*. Accessed: 2025-09-19. The Khronos Group Inc. 2025. Chap. 13 Images. URL: <https://registry.khronos.org/vulkan/specs/latest/html/vkspec.html#images>.
- [Lun25] C. Lunarg. *vk-bootstrap: Vulkan Bootstrapping Library*. accessed: 2025-09-19. 2025. URL: <https://github.com/charles-lunarg/vk-bootstrap>.
- [ric23] ricecookie. *What is Premake?* boh. 2023.
- [Whi80] T. Whitted. “An improved illumination model for shaded display”. In: *Commun. ACM* 23.6 (June 1980), pp. 343–349. ISSN: 0001-0782. DOI: 10.1145/358876.358882. URL: <https://doi.org/10.1145/358876.358882>.